

Modelo de Dados Proposto - PU-MVP-1.0

Estado atual implementado

Tabelas existentes

- `users`: autenticação básica (admin), perfil e credenciais.
- `accounts`: contas para pagamento consolidado (título, valor, vencimento).
- `payments`: consolidação de várias contas em um pagamento único.

Lacunas vs. novas regras e history board

- Não existe estrutura de `categorias` e `sub-categorias`.
- Não existe estrutura de `benefícios` e parceiros.
- Não existe modelagem explícita para `DDA` (contas vindas por débito direto autorizado).
- Dashboard atual não possui agregações por categoria, período, status e benefícios.
- Não existe trilha de operação/compliance (status operacional por conta e conciliação).

Modelo de dados alvo (v1.0)

1) users

- id (uuid, pk)
- email (varchar, unique)
- name (varchar)
- login_username (varchar, unique)
- password_hash (varchar)
- role (varchar)
- created_at, updated_at (timestamp)

2) categories

- id (uuid, pk)
- user_id (uuid, fk -> users.id)
- name (varchar)
- monthly_budget (decimal, null)
- color (varchar, null)
- active (boolean)
- created_at, updated_at

3) subcategories

- id (uuid, pk)
- user_id (uuid, fk -> users.id)
- category_id (uuid, fk -> categories.id)
- name (varchar)
- monthly_budget (decimal, null)
- active (boolean)
- created_at, updated_at

4) accounts

- id (uuid, pk)
- user_id (uuid, fk -> users.id)
- category_id (uuid, fk -> categories.id, null)
- subcategory_id (uuid, fk -> subcategories.id, null)
- source_type (enum: boleto, dda, card_invoice, rent, utility, health, other)
- title (varchar)
- amount (decimal)
- due_date (date)
- barcode_line (varchar, null)
- external_ref (varchar, null)
- status (enum: open, scheduled, paid, overdue, canceled)

- notes (text, null)
- created_at, updated_at

5) payments

- id (uuid, pk)
- user_id (uuid, fk -> users.id)
- total_amount (decimal)
- status (enum: pending, processing, paid, failed, canceled)
- method (enum: boleto_unico, pix, card, debit)
- provider (varchar, null)
- provider_ref (varchar, null)
- boleto_line (varchar, null)
- paid_at (timestamp, null)
- created_at, updated_at

6) payment_items

- id (uuid, pk)
- payment_id (uuid, fk -> payments.id)
- account_id (uuid, fk -> accounts.id)
- amount (decimal)
- created_at

7) benefits

- id (uuid, pk)
- user_id (uuid, fk -> users.id)
- name (varchar)
- benefit_type (enum: cashback, seguro_vida, seguro_residencial, seguro_auto, saude, parceiro)
- partner_name (varchar, null)
- monthly_cost (decimal, null)
- estimated_return (decimal, null)
- status (enum: active, inactive, pending)
- details (text, null)
- created_at, updated_at

8) dashboard_snapshots (opcional para performance)

- id (uuid, pk)
- user_id (uuid, fk -> users.id)
- period_ref (varchar) ex: 2026-03
- total_open_amount (decimal)
- total_paid_amount (decimal)
- category_breakdown (jsonb)
- benefits_breakdown (jsonb)
- generated_at (timestamp)

Relacoes principais

- `users` 1:N `categories`
- `categories` 1:N `subcategories`
- `users` 1:N `accounts`
- `accounts` N:1 `categories` e `subcategories`
- `users` 1:N `payments`
- `payments` 1:N `payment\_items`
- `payment\_items` N:1 `accounts`
- `users` 1:N `benefits`

Prioridade de implementacao recomendada

1. `categories`, `subcategories` e vinculo em `accounts`.
2. `benefits`.

3. `payment\_items` para eliminar dependencia de `accountIds` em json.
4. agregacoes de `dashboard`.
5. trilha operacional/compliance (auditoria e conciliacao).